

The Study Almanac

A web application that generates personalized AI study plans using fuzzy-logic reasoning and large language models - wrapped in a vintage archival interface.

STACK: REACT 19 / VITE / TYPESCRIPT / TAILWIND CSS 4 / CONVEX (DB + AUTH + ACTIONS) / BUN
SEPTEMBER 2026

VINTAGE EDITION

01 Overview

The Study Almanac lets a user sign up with an email, fill a short study brief (subject, exam date, hours per week, prior knowledge), watch a live fuzzy-inference preview as they type, and compose a week-by-week study plan that is filed under their own account.

Two engines cooperate: a 13-rule Mamdani fuzzy-logic system decides intensity, review cadence and sitting length from the brief, and a large language model writes the actual week-by-week sessions from that verdict. If the model is unavailable or its answer cannot be parsed, a deterministic composer produces a complete plan from the same fuzzy verdict - the app never fails open.

02 Features

- **Email sign-in only.** Six-digit email OTP via Convex Auth. One method, exactly as specified for version 1.
- **Live fuzzy inference.** Fuzzification chips, three defuzzified gauges, sitting estimate, and a full rule trace with membership strengths (μ) - recomputed on every slider move.
- **AI plan composition.** A Convex node action sends the brief plus the fuzzy verdict to an LLM, which writes titles, focus lines, sessions (day / minutes / kind) and checkpoint milestones.
- **Guaranteed fallback.** If no AI key exists or the model misbehaves, the deterministic Almanac engine composes the plan from the same verdict. No placeholder states.
- **Filed archive.** Every plan is saved with ownership checks; reopen or withdraw past folios at any time.
- **Print-friendly folios.** Navigation chrome is excluded from print; a plan prints cleanly as a study document.

03 Architecture and pipeline

```
brief (subject, date, hours, knowledge)
  |
  v
infer() . src/lib/fuzzy.ts      <- runs in-browser (live) and in Convex
fuzzify -> 13 rules (min) -> aggregate (max) -> centroid -> 3 outputs
  |
  v
generatePlan action . src/convex/generatePlan.ts
+- platform AI integration (auto-configured key) -> LLM writes the weeks
+- else OPENAI_API_KEY / OPENAI_BASE_URL set -> direct compatible request
+- otherwise / on error -> buildAlmanacPlan() (src/lib/compose.ts)
  |
  v
savePlan mutation -> `plans` table -> folio display + archive
```

Layer	Technology
Frontend	React 19, Vite 7, TypeScript, Tailwind CSS 4, shadcn/ui, Framer Motion, React Router
Backend and database	Convex - schema, reactive queries, mutations, "use node" actions
Authentication	Convex Auth with an email OTP provider (six-digit code, 15-minute expiry)
Reasoning	Mamdani fuzzy inference (src/lib/fuzzy.ts) + LLM composition with deterministic fallback
Tooling	Bun package manager, ESLint, Prettier

04 The fuzzy-logic engine

Inputs are fuzzified with triangular and trapezoidal membership functions, fired through 13 hand-written linguistic rules using min-implication, aggregated with max, and defuzzified by centroid over 0-100 universes.

Input	Linguistic terms
Hours per week (1-60)	scarce / moderate / ample
Days until exam (1-365)	urgent / near / distant
Prior knowledge (0-10)	weak / working / strong
Output (0-100 universe)	Linguistic terms
Intensity	gentle / steady / rigorous

Output (0-100 universe)	Linguistic terms
Review cadence	rare / weekly / dense
Session length	short / medium / long -> crisp minutes (20-90)

Example rule: IF hours = moderate AND exam = near AND knowledge = working THEN intensity = steady, cadence = weekly, session = medium.

The exact same infer() function runs in the browser for the live preview and inside the Convex action for the authoritative plan, so what the user sees while dragging the sliders is exactly what gets filed.

05 AI composition and fallback

- The compose action builds a strict JSON prompt: exact week count, session count, session minutes, and the fuzzy verdict.
- The model's response is structurally sanitised: exact week count, sessions-per-week bounds, minutes clamped to 10-180, session kinds whitelisted to study / practice / review / mock, string length caps.
- Any failure (no key, timeout, unparseable JSON) falls back to the deterministic engine; the folio is stamped "LLM" or "Almanac engine" accordingly.
- Keys are read server-side only, inside the "use node" action via process.env - they are never shipped to the client.

LLM path tried in order:

1. platform AI integration (VLY_INTEGRATION_KEY, pre-configured)
2. direct OpenAI-compatible endpoint (OPENAI_API_KEY + optional OPENAI_BASE_URL / OPENAI_MODEL, default gpt-4o-mini)
3. deterministic Almanac engine (foundation -> build -> taper phases, final sitting always a full rehearsal)

06 Authentication

- Email OTP only (six-digit key, 15-minute validity) via Convex Auth - the sole sign-in method in version 1.
- Signed-out visitors hitting a protected route see an explanation and are sent to /auth?returnTo=...; after sign-in they return to the intended page.
- /dashboard is wrapped in RequireAuth; sign-in redirects to /dashboard.
- Plan mutations enforce ownership server-side: savePlan, listMine and remove all resolve the signed-in user's ID.

07 Data model

```

plans
  userId      Id<"users">      (index: by_user)
  inputs      { subject, examDate, hoursPerWeek, priorKnowledge,
                daysUntilExam, notes? }
  fuzzy       { intensity, cadence, session, sessionMinutes,
                labels{intensity,cadence,session}, trace[{text,strength}] }
  weeks[]     { index, title, focus,
                sessions[{ day, task, minutes, kind }], milestone }
  composedBy  "llm" | "almanac"
  createdAt   number

users (Convex Auth) name?, image?, email?, emailVerificationTime?,
                   isAnonymous?, role?      (index: email)

```

08 Local setup

Requirements: Bun (<https://bun.sh>) and a Convex project - the template ships configured.

```

bun install      # install dependencies
bun convex dev   # start Convex + codegen, then in another terminal:
bun run dev      # start the Vite dev server

```

Routes: / landing page /auth sign-in /dashboard protected workspace.

09 Environment variables

Variable	Where	Req.	Purpose
VITE_CONVEX_URL	.env.local (platform-set)	yes	Convex client URL
VLY_INTEGRATION_KEY	set at project creation	yes	Platform AI integration for LLM composition
OPENAI_API_KEY	Convex deployment env	no	Optional own OpenAI-compatible endpoint
OPENAI_BASE_URL	Convex deployment env	no	Custom base URL (default api.openai.com/v1)
OPENAI_MODEL	Convex deployment env	no	Model id (default gpt-4o-mini)

LLM composition works out of the box via the platform integration. If every AI path fails the app still works end-to-end - plans are composed by the deterministic Almanac engine and stamped accordingly.

10 Deployment

- **Backend:** `bun convex deploy` pushes functions and schema; set any optional `OPENAI_*` variables in the deployment environment.
- **Frontend:** `bun run build` outputs `dist/` - host on any static provider (Render, Netlify, Vercel, GitHub Pages) with `VITE_CONVEX_URL` pointing at the deployed Convex deployment. Configure an SPA rewrite of all routes to `/index.html`.
- **Live demo:** the hosted Freebuff preview of this project is the live demo - sign in at `/auth` and compose a plan at `/dashboard` immediately.

Streamlit Community Cloud and Hugging Face Spaces host Python / Streamlit / Gradio apps and cannot run this Vite + Convex stack. The correct free hosts are the built-in live preview (demo) and Render / Netlify / Vercel-style static hosts (production), with Convex as the backend.

11 Scripts and checks

Command	Purpose
<code>bun run dev</code>	Vite dev server
<code>bun run build</code>	Typecheck + production build -> <code>dist/</code>
<code>bun run lint</code>	ESLint
<code>bun run format</code>	Prettier
<code>bun convex dev --once && bun run dev --noEmit</code>	Once-bun-convex-codegen + TypeScript typecheck

12 Project structure

```

src/
  lib/
    fuzzy.ts          # Mamdani engine (MFs, 13 rules, centroid)
    compose.ts        # Deterministic Almanac composer + date helpers
    plan.ts           # Shared plan types + summarize()
    validators.ts      # Convex validators mirroring the plan types
  convex/
    schema.ts         # auth tables + plans (indexed by user)
    plans.ts          # savePlan / listMine / remove (ownership enforced)
    generatePlan.ts    # "use node" action: verdict -> LLM or fallback
  components/
    StudyPlanForm.tsx  # intake brief
    InferenceLedger.tsx # live fuzzification / gauges / rule trace
    PlanFolio.tsx      # folio header + week cards (print-friendly)
    vintage.tsx        # archival primitives (rules, stamps, gauges)
    RequireAuth.tsx    # protected-route wrapper with returnTo
  pages/
    Landing.tsx        # hero, method, contents, specimen, CTA
    Dashboard.tsx      # protected workspace: form + ledger + archive
    Auth.tsx           # email OTP sign-in
  index.html           # fonts + title
  src/index.css        # vintage theme tokens + paper grain
  README.md            # full setup / deployment instructions
  docs/documentation.html # print-ready HTML documentation
  docs/documentation.pdf # this document

```

13 Deliverables and links

Deliverable	Location
Complete source code	This repository - export via the Freebuff / Vly project UI (version control is managed by the platform)
GitHub repository	github.com/sagar777-sys - create the repo, then push this source (see README)
Live demo link	The hosted Freebuff preview of this project - publish via freebuff.com/project/<name>?publish=true for a stable URL
Documentation PDF	docs/documentation.pdf (this file). Regenerate with: node docs/generate-pdf.mjs
Documentation source	docs/documentation.html - open in a browser and use Print / Save as PDF
Setup instructions	README.md and sections 08-10 of this document

Security note: never share account passwords in chat or commit them to a repository. Rotate any credential that has been exposed, store secrets in a password manager, and enable two-factor authentication. No proprietary libraries, keys, or secrets are embedded in this repository.